100 Days of Azure DevOps

Day 17 Handout — Shared repo vs forks & least privilege

Learning in public · Personal lab only · Educational content · Not a sales pitch

Architecture A — contribution models

Model	How contribution works	Best for
Shared repo	Feature branches + PRs in one Azure Repos project	Internal teams with shared ownership
Fork workflow	Work in fork → PR back to upstream	OSS, vendors, external collaborators

Architecture B — Azure Repos permission bands

Role	Typical power	Danger if over-granted
Readers	View code and PRs	Low — usually safe default
Contributors	Push branches, create PRs	Force push / bypass policies
Project Admins	Repo settings + security	Wide-open permissions drift

One-liner to remember

Access is a production control · If everyone is Admin, nobody is protected

Step-by-step lab (20–30 min)

In **azure-100-labs** (Azure Repos):

1. Open **Project settings** → **Repositories** → **Security**
2. Review Readers / Contributors / Project Admins permissions
3. Confirm Contributors: Contribute allowed; Force push denied for main
4. Create branch: **git switch -c feature/day17-repo-permissions**
5. Add **docs/repo-permissions.md** ADR (shared-repo vs fork + baseline table)
6. Commit, push, and open a Pull Request

Permissions to treat as incident-class

```
• Force push (especially on main)
• Bypass policies when completing pull requests
• Delete repository
• Manage permissions (who can change who)
```

Done checklist

- I can explain fork vs shared-repo workflows
- Reviewed Repository Security in Azure Repos
- Permissions ADR documented in docs/repo-permissions.md
- Force-push risk on main understood

Tomorrow — Day 18

Migrating repos to Azure Repos — moving history without losing trust.